PES UNIVERSITY
School of Data Sciences
MTech in Data Science

# LensRAG - Long-range ENriched Search for RAG

## Corpus-Level Aggregation Queries over Enterprise Documents

Team Project Report

Capstone Project | Batch A, January 2025

**mTechPesJan2025AGrp5**



| Team member | Role | Roll No. |
|---|---|---|
| Alok Kumar | Research design | PES2PGE25DS005 |
| Bhibu Prasanna Mohapatra | Research design | PES2PGE25DS014 |
| Gautam Pandey | Research design | PES2PGE25DS022 |
| Raghavendra Rajarao | Research design | PES2PGE25DS069 |
| Shivakumar Athreya | Research design | PESPGE2025DS084 |

**Faculty Advisor:** Dr. Narayana Darapaneni & Anwesh Reddy Paduri


**Code & data repository:** `https://github.com/brags-labs/corpusrag`

December 2026 Bangalore

# Abstract

Write 250–300 words. State the problem and why it matters to industry (cite the market driver), the gap in prior work, what you built or measured, the headline quantitative result (e.g., “the gateway reduced tool-poisoning success from 61% to 4% at a 7% task-completion cost”), and the released artifact (benchmark, dataset, reference architecture).

**Keywords:** agentic AI; Model Context Protocol; retrieval-augmented generation; evaluation; LLMOps.

# Declaration and AI-Use Disclosure

We declare that this report is our own work, that all sources are cited, and that all experiments reported were actually executed as described.

**Generative-AI tools used.** List every tool (e.g., Claude, ChatGPT, Copilot) and for what purpose (‘literature search, code drafting, proofreading, figure generation). State which parts of the text or code were AI-drafted and how they were verified. Per programme policy, AI tools may not be used to fabricate data, results or references.

**Data and ethics.** No PII information is used., These are HR policy related mock data.

# Acknowledgements

Optional.

# Contents

**Abstract** i

**Declaration and AI-Use Disclosure** ii

**Acknowledgements** iii

**Abbreviations** ix

**1 Introduction** 1

1.1 Context and Motivation . . . 1

1.1.1 Recommended Approach . . . 2

1.2 Problem Statement . . . 5

1.3 Research Questions and Hypotheses . . . 6

1.4 Scope and Assumptions . . . 6

1.5 Contributions . . . 7

1.6 Report Organisation . . . 7

**2 Background and Related Work** 8

2.1 Foundations . . . 8

2.1.1 Retrieval-Augmented Generation . . . 8

2.1.2 Known Limitations of Standard RAG . . . 8

2.1.3 Local versus Global Queries . . . 9

2.2 Prior Work . . . 9

2.2.1 Advanced and Adaptive RAG . . . 9

2.2.2 Corpus Organisation, Chunking and Context-Enriched Embeddings . . . 9

2.2.3 Corpus Partitioning and Domain Scoping . . . 10

2.2.4 Agentic RAG and Query Decomposition . . . 10

2.2.5 Retrieval Quality, Coverage and Evidence Selection . . . 11

2.2.6 Structured and Graph-Based RAG . . . 11

2.2.7 Knowledge Refinement and Structured Reasoning . . . 12

2.2.8 Evidence Verification and Conflict Handling . . . 13
2.2.9 Corpus-Level Aggregation: Benchmarks and Methods . . . 13
2.2.10 LLM-Powered Analytics over Unstructured Documents . . . 14
2.2.11 Long-Context Models as an Alternative . . . 14
2.3 Comparative Discussion . . . 14
2.4 Gap Analysis . . . 15
2.5 Positioning of This Work . . . 15

**3 Methodology** **18**
3.1 Research Design . . . 18
3.2 System Architecture . . . 18
3.3 Data and Benchmarks . . . 18
3.4 Evaluation Protocol . . . 20
3.5 Cost, Governance and Observability Instrumentation . . . 20
3.6 Threats to Validity . . . 20

**4 Implementation** **21**
4.1 Development Environment and Reproducibility . . . 21
4.2 Key Components . . . 21
4.3 Deployment . . . 21

**5 Results** **22**
5.1 Main Findings . . . 22
5.2 Ablations and Sensitivity . . . 22
5.3 Cost–Performance Trade-offs . . . 22
5.4 Error Analysis . . . 22

**6 Discussion** **23**
6.1 Interpretation . . . 23
6.2 Implications for Practitioners . . . 23
6.3 Implications for Research . . . 23
6.4 Limitations . . . 23
6.5 Ethical, Legal and Societal Considerations . . . 23

**7 Conclusion and Future Work** **24**

7.1 Summary of Contributions . . . . . 24

7.2 Future Work . . . . . 24

7.3 Publication Plan . . . . . 24

**A Individual Contribution Statement** **30**

**B Weekly Progress Log** **31**

**C Reproducibility Checklist** **32**

**D Prompts, Judge Instructions and Additional Results** **33**

# List of Figures

1.1 Step-by-step view of the LensRAG query pipeline. Tags show the model used at each stage; aggregation is computed in code rather than by the LLM. . . . . 4

1.2 Context-level architecture of the proposed LensRAG method. Dashed arrows show retrieval from the context-enriched index; no SQL schema is built over the corpus. . . . . 5

3.1 System architecture. . . . . 18

# List of Tables

2.1 Positioning of representative prior work against the requirements of this project. Entries summarise each work's primary focus as described by its authors. . . . . 16

3.1 Candidate models for each stage of the LensRAG query pipeline. Final choices and exact versions are reported in Chapter 4. . . . . 19

5.1 Main results (illustrative). Best value per column in bold. . . . . 22

# Abbreviations

| | |
|---|---|
| A2A | Agent2Agent protocol |
| ADLC | Agent development life cycle |
| HITL | Human-in-the-loop |
| LLMOps / AgenticOps | Operations practices for LLM- and agent-based systems |
| MCP | Model Context Protocol |
| RAG / CRAG | Retrieval-augmented generation / corrective RAG |

# 1 Introduction

## 1.1 Context and Motivation

The objective of this research is to develop a novel architecture and algorithm for Retrieval-Augmented Generation (RAG) that reliably and accurately answers corpus-level queries, such as counting, summarising themes or tracking sentiment shifts, without converting the documents into structured SQL tables.

Standard RAG systems are good at answering specific queries, where the answer can be found in one or a few documents. However, they perform poorly on questions that require collating information by scanning all the documents in the corpus, for example "How many documents mention a data breach?" or "What are the main themes across all reviews?". Luo et al. [26] demonstrate that existing RAG architectures perform poorly on such global queries, producing fluent and confident but factually wrong outputs. This failure is especially critical in high-stakes domains such as healthcare, law and finance, where global reasoning over entire document collections carries significant decision-making value.

Ongoing research in this field has shown improvements on aggregation queries. Structured RAG (S-RAG) converts the documents into database records while they are being indexed, so that a question can later be answered by an SQL query [19]. SParC-RAG (Adaptive Sequential–Parallel Scaling with Context Management) takes a different route, mixing step-by-step refinement with several sub-queries explored in parallel, all sharing one managed context [43]. These results highlight an opportunity for further research. The present work aims to identify improvements during embedding and vector-based retrieval by including contextual information that supports long-range contextualisation, with the potential to overcome current RAG limitations on aggregation queries.

The research is grounded in the enterprise human-resources (HR) domain. Employees and HR staff routinely ask cross-cutting questions about entitlements across countries, employee categories and policy versions, and policy corpora contain features that make such questions difficult: country-specific addenda, exceptions attached to general rules, cross-references between chapters, and superseded versions of the same policy that remain in circulation. We therefore construct an HR policy corpus for a fictional multinational, BragsLabs. Its core is a 47-chapter policy manual (about 41,000 words and 130 tables) covering global policies, cross-border rules and addenda for ten countries, extended with earlier policy versions, circulars, and synthetic helpdesk tickets, grievances and exit notes. A local question on this corpus, such as "How many days of paternity leave do employees in India receive?", is answered by a single section of the India

addendum. A global question, such as "How many countries offer more than 20 days of annual leave?" or "Which three countries have the longest notice period?", requires finding the relevant clause in every country addendum, extracting comparable values and computing an exact count or ranking. Thematic and sentiment questions, such as "What are the most common themes in grievances raised this year?" or "How has sentiment in exit notes changed since the revised leave policy?", require reading across hundreds of free-text records. The research proceeds in three stages:

1. **Baseline.** Build a domain-specific test set of local and aggregation queries over the BragsLabs corpus, with ground-truth answers derived independently of the systems under test, and run standard top-$k$ RAG and long-context prompting on it. This documents where and how the baselines fail and provides the comparative reference for later stages.
2. **Architecture and algorithm.** Design a retrieval and aggregation method (Section 1.1.1) that enriches each chunk with its document context before embedding, retrieves for coverage rather than only relevance, and computes aggregate answers deterministically at query time, without building a SQL schema over the corpus.
3. **Evaluation.** Compare the proposed method with the baselines, and with published approaches such as GlobalRAG [26] where feasible, on the same test set and on progressively larger versions of the corpus.

The performance evaluation will show whether the proposed method improves answers to corpus-level queries over standard RAG baselines. The evaluation framework will include an aggregation accuracy metric (correctness of the aggregated answer), a hit rate or retrieval recall metric (the share of required evidence that was retrieved), and a faithfulness rate (the share of statements in the generated response that are supported by facts in the corpus). It will also report token cost per query and latency, giving a comprehensive performance report on the sample datasets.

### 1.1.1 Recommended Approach

We propose *LensRAG* (Long-range ENriched Search for Retrieval-Augmented Generation). The method keeps the corpus as text and a vector index, as in standard RAG, and changes three things: what is embedded, how much is retrieved, and where the final computation happens. Unlike S-RAG [19], it does not define a schema or load the corpus into a database at ingestion. The only structured data are short-lived evidence records created for a single query, each pointing to the passage it came from.

**Ingestion: context-enriched indexing.** Each document is parsed into a tree of chapters, sections and tables, so that tables are not split across chunks. Before a chunk is embedded, a short contextual header is prepended to it: the document title, the section path (for example *HR Manual* ⟩ *Ch. 40 India Addendum* ⟩ *40.3 Leave*), the scope (country, employee category), the version and effective date, and the entities it mentions. This gives each chunk long-range context that is otherwise lost when a document is split, so that a clause such as "this applies for 10 days" can be matched to the right country and policy. Chunks are indexed in a hybrid index (BM25 and dense embeddings), and their metadata in an entity list that records, for example, every country that has an addendum.

**Query time.** Figure 1.1 summarises the query pipeline. A router classifies the query. Local queries follow the standard retrieve–rerank–generate path. For global queries, the query is expanded into one sub-query per relevant entity, taken from the entity list, and retrieval continues until no new relevant entities are found, rather than stopping at a fixed $k$. An LLM then checks each candidate passage against the query condition and extracts the required value with its supporting evidence span. Counts, extrema, rankings and top-$k$ lists are computed in code over these records. Theme and sentiment queries instead use a map–reduce step: each passage is labelled with themes or sentiment, labels are grouped (for sentiment shift, by time period), and the groups are summarised. Finally, a verification step checks completeness against the entity list, flags conflicting values from different sections or policy versions, and confirms that every statement in the answer is backed by an evidence span. The method is model-agnostic: each stage in Figure 1.1 names the role of the model it needs, and candidate open-source and API models for each role are listed in Table 3.1 (Section 3.2).

Figure 1.2 shows the method at context level: the users, the document sources and the two query paths.

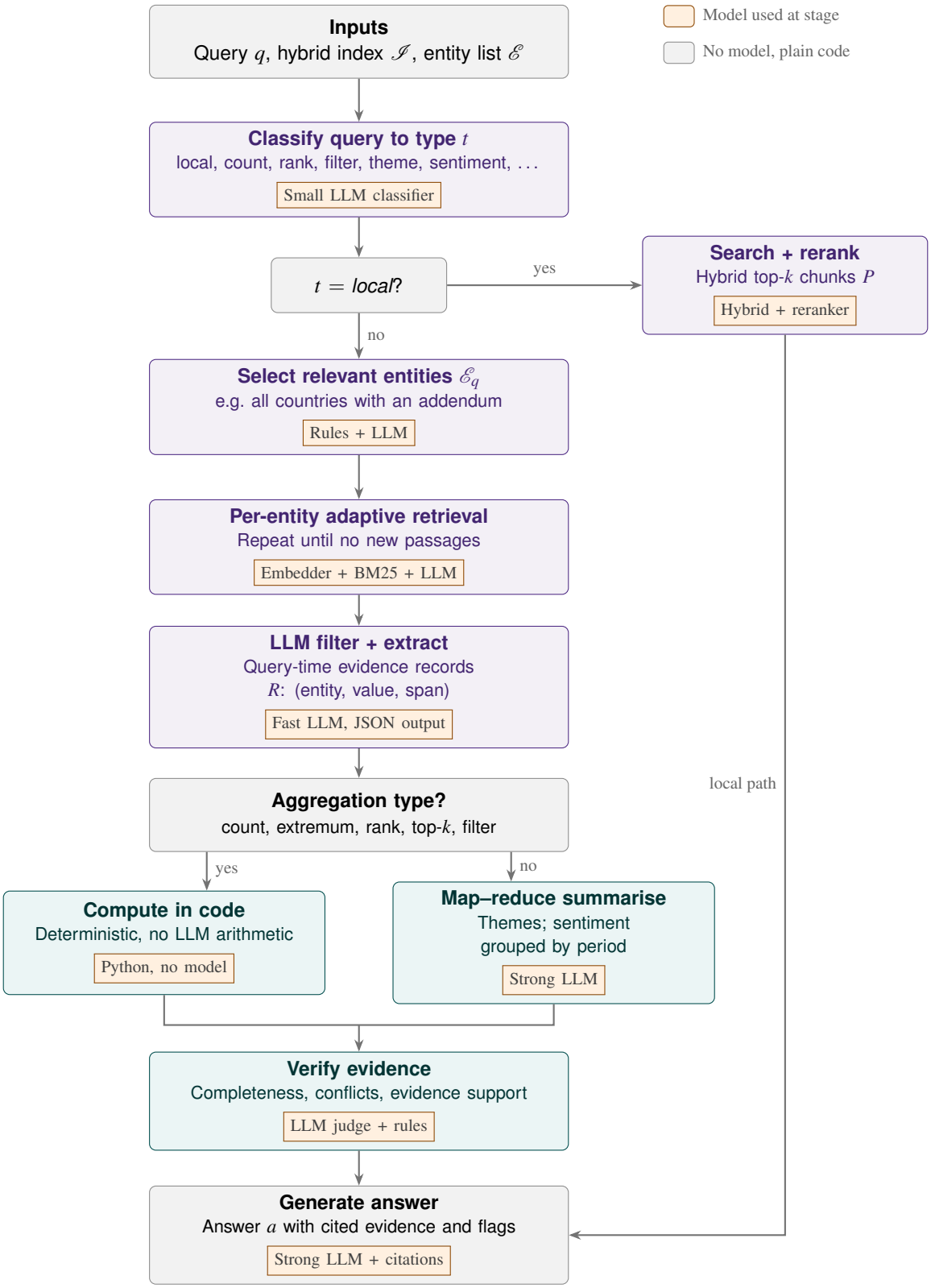



**Figure 1.1.** Step-by-step view of the LensRAG query pipeline. Tags show the model used at each stage; aggregation is computed in code rather than by the LLM.

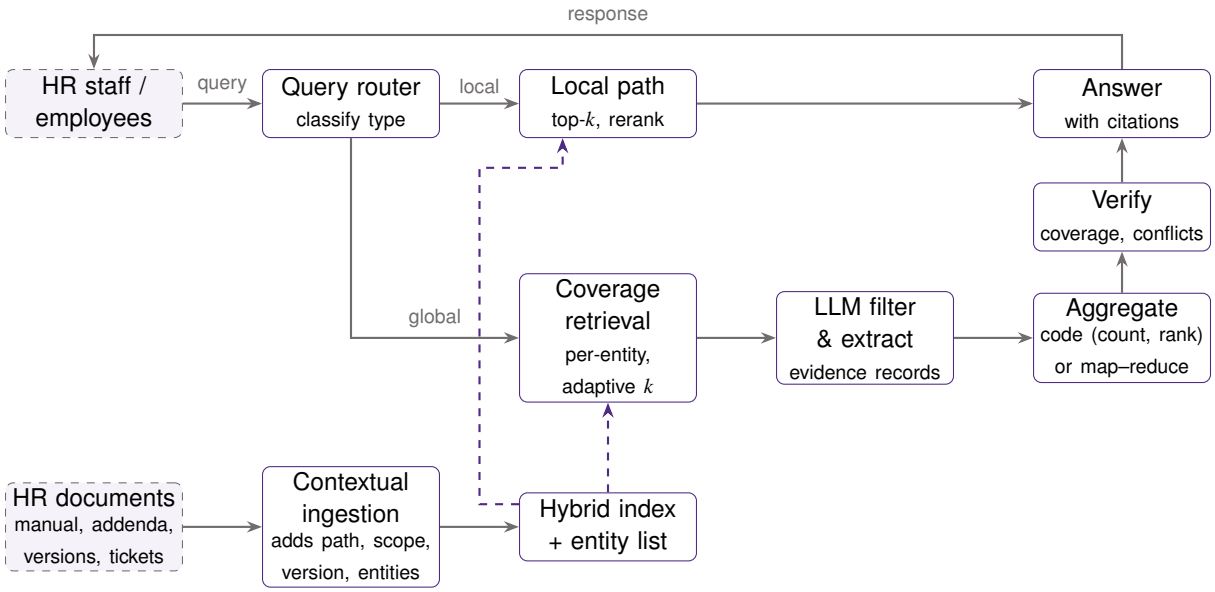



**Figure 1.2.** Context-level architecture of the proposed LensRAG method. Dashed arrows show retrieval from the context-enriched index; no SQL schema is built over the corpus.

## 1.2 Problem Statement

Standard RAG performs poorly when responding to an aggregation or a corpus-level query – those that require reasoning over entire corpus rather than responding from top-ranked chunks. Query responses from standard RAG may be factually incorrect on aggregation queries, highlighting the need for effective and evidence-based interventions. Although multiple research projects have attempted to solve this problem, there is still an opportunity to address and measurably bridge the gaps. This project investigates approaches to solve this problem, and we aim to improve the responses to aggregation queries on baselined datasets, paving the way for further research in this area.

To address this problem, the project uses the enterprise HR domain as its baselined dataset. We build an HR policy corpus for a fictional multinational, BragsLabs. It consists of a 47-chapter policy manual with ten country addenda, extended with earlier policy versions and related documents. On this corpus we create a set of specific and aggregation queries with verifiable answers. We then measure how standard RAG responds to them, which gives the baseline. Next, we design an approach that routes each query to a suitable strategy, gathers evidence from across the whole corpus rather than only from top-ranked chunks, and computes aggregate answers exactly. Finally, we evaluate the improvement in responses to aggregation queries against this baseline. Although the evaluation uses HR documents, the approach avoids a fixed, domain-specific schema, so it can be applied to other document collections and support further research in this area

## 1.3 Research Questions and Hypotheses

**RQ1.** How well do standard RAG and long-context prompting answer global questions (counting, extremum, sorting, top-$k$, filtering, themes and sentiment) over an enterprise HR policy corpus, compared with local questions?

**RQ2.** Does context-enriched, coverage-oriented retrieval with query-time aggregation answer global questions more accurately than standard RAG, long-context prompting and a GlobalRAG-style baseline, and at what cost in tokens and latency?

**RQ3.** Can explicit verification steps detect incomplete answers and conflicts between policy sections or versions?

The corresponding hypotheses are:

**H1.** Standard top-$k$ RAG scores significantly lower on global questions than on local questions. Long-context prompting narrows the gap when the whole corpus fits in the context window, but degrades as the corpus is scaled beyond it.

**H2.** The routed pipeline achieves higher set-level F1 and exact-match accuracy on global questions than all baselines, without reducing accuracy on local questions

**H3.** Completeness and conflict checks flag a majority of incomplete or conflicting answers with a low false-alarm rate.

## 1.4 Scope and Assumptions

- **Corpus.** The core corpus is a single fictional company's HR policy manual (version 3.0, effective 1 November 2026): 47 chapters, about 475 sections, about 41,000 words (roughly 55,000 tokens) and about 130 tables. Chapters 1–35 cover global policies, Chapter 36 cross-border rules, Chapter 37 country comparison tables and Chapters 38–47 addenda for ten countries. Using a fictional company avoids confidentiality and personal-data issues.
- **Corpus scaling.** Because the manual alone fits in current long-context windows, the corpus is extended with earlier versions (v1.0 and v2.0) and related documents such as circulars and synthetic helpdesk tickets, grievances and exit notes, so that the effect of corpus size can be measured.
- **Shortcut control.** Chapter 37 already aggregates many cross-country comparisons, which would turn global questions into local ones. It is excluded from the retrieval corpus and used only to check gold answers.
- **Out of scope.** Multimodal content (images, scanned handwriting), real employee data, multilingual documents and production deployment.

## 1.5 Contributions

- **Benchmark:** an HR-policy global QA test set covering local, counting, extremum, sorting, top-$k$, filtering, comparison, theme, sentiment and conflict-detection questions, with ground truth derived independently of the systems under test.
- **Method:** LensRAG, a schema-free global RAG method that enriches chunks with document context before embedding, retrieves for coverage, aggregates in code at query time and verifies completeness and consistency.
- **Empirical findings:** a comparison of standard RAG, long-context prompting, a GlobalRAG-style baseline and the proposed method on aggregation accuracy, retrieval recall, faithfulness, cost and latency, including how results change as the corpus grows.
- **Practitioner guidance:** recommendations on when global questions need more than standard RAG, and which strategy suits which question type.

## 1.6 Report Organisation

Chapter 2 reviews background and related work on RAG, agentic and structured RAG, corpus-level aggregation and LLM-based document analytics, and identifies the gaps this project addresses. Chapter 3 describes the methodology: corpus construction, test-set design, system architecture and evaluation protocol. Chapter 4 covers implementation. Chapter 5 presents results, ablations, cost–performance trade-offs and error analysis. Chapter 6 discusses implications, limitations and ethical considerations, and Chapter 7 concludes with future work.

# 2

# Background and Related Work

This chapter reviews the background needed to follow the proposed method and the prior work it builds on. Section 2.1 introduces RAG, its known limitations and the distinction between local and global (aggregation) queries. Section 2.2 groups prior work by its principal contribution and relates each group to the design choices made in LensRAG (Section 1.1.1): how the corpus is organised and partitioned, how chunks are contextualised, how retrieval achieves coverage, how answers are aggregated, and how evidence is checked. The groups are not mutually exclusive, since several systems combine hierarchy, graph structure, retrieval control and evidence assessment. Section 2.3 compares the groups, Section 2.4 identifies the gaps this project addresses, and Section 2.5 positions the proposed method.

## 2.1 Foundations

### 2.1.1 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) couples a parametric language model with a non-parametric document store: a retriever selects passages relevant to the query, and the generator conditions its answer on them. The original formulation combined a neural retriever with a sequence-to-sequence generator and explored two variants, RAG-Sequence and RAG-Token, establishing the principle that generation can be conditioned on retrieved external evidence rather than relying entirely on parametric knowledge [20]. RAG has since become the default way to ground large language models (LLMs) in private or frequently changing knowledge, because it reduces hallucination and allows sources to be updated without retraining [9]. A typical pipeline splits documents into fixed-size chunks, embeds them, retrieves the top-$k$ most similar chunks for a query, and passes them to the LLM as context. The corpus-level approaches reviewed in this chapter extend that principle by changing how the external collection is structured, searched and checked.

### 2.1.2 Known Limitations of Standard RAG

Practitioner and survey studies have catalogued recurring failure points in this pipeline [5, 9]. On the retrieval side, relevant content may be missed because of vocabulary mismatch, poor chunk boundaries that separate conditions from their exceptions, or a top-$k$ cut-off that is too small. A chunk also loses the context of the document it came from: a clause such as “this applies for 10 days” no longer says which country, policy or

version it belongs to. On the generation side, models may ignore or misread retrieved evidence, are sensitive to where evidence appears in a long context [24], and rarely flag contradictions between sources. These limitations are tolerable for *local* questions, whose answers lie in one or a few passages. They become decisive for *global* questions, whose answers depend on evidence spread across the whole corpus.

### 2.1.3 Local versus Global Queries

Edge et al. [7] first drew the distinction between local and global questions, observing that standard RAG fails on corpus-wide questions such as "What are the main themes in this dataset?", which are closer to query-focused summarisation than to retrieval. Luo et al. [26] sharpen the definition: *global RAG* requires aggregating and analysing information across an entire collection, as in counting, extremum, sorting and top-$k$ queries. In this setting a retriever must find *all* relevant items rather than the *most* relevant ones, and the answer must be computed exactly rather than paraphrased. This report uses the term *aggregation queries* for both kinds: those that need an exact computation (count, extremum, rank, top-$k$, filter) and those that need a corpus-wide summary (themes, sentiment shift).

## 2.2 Prior Work

### 2.2.1 Advanced and Adaptive RAG

A first line of work improves the retrieve-then-generate loop itself. FLARE triggers retrieval when generation confidence is low [17], IRCoT interleaves retrieval with chain-of-thought reasoning for multi-hop questions [39], and Self-RAG learns to decide whether external retrieval is needed at all [4]. These methods make retrieval less dependent on a single fixed decision at the start of the pipeline and improve robustness for local and multi-hop questions. They still assume that a small evidence set is sufficient, so they do not address the completeness requirement of aggregation queries. Their evidence-assessment mechanisms are discussed in Section 2.2.8.

### 2.2.2 Corpus Organisation, Chunking and Context-Enriched Embeddings

Because fixed-size chunking strips each passage of its surrounding context, several methods restore that context or organise the corpus into levels of abstraction. Late chunking embeds the full document with a long-context embedding model and only then pools token embeddings into chunk vectors, so each chunk vector reflects the document around it [10]. RAPTOR groups semantically related text units, summarises each group and repeats the process to build a tree of increasingly broad summaries, so retrieval can draw on detailed passages or on summaries that represent a wider context [34]. RAPTOR established an influential approach to multi-resolution retrieval, but a tree does not by itself express every relationship between branches or between separate documents. Ψ-RAG applies the same idea across documents: its tree of abstracts can be restructured by merging and collapsing nodes, and an agent decides at each step whether to

read a broad abstract or descend to detailed text [46]. The central issue it addresses is granularity: a system needs a compact view of the corpus for orientation, but must be able to return to the underlying evidence when a question requires detail.

Chunking choices also matter empirically. Maharjan and Yadav [28] evaluate RAG configurations on public-health policy documents, comparing a language model without retrieval, a basic semantic-search pipeline and a pipeline that adds cross-encoder reranking. They report higher faithfulness with retrieval and a further improvement with reranking, and show that retrieval quality depends on linked choices, including how source documents are segmented and how candidates are ordered. This result is directly relevant to the policy domain studied here. LensRAG follows the same structure-aware view of chunking: it splits documents along their section structure, keeps tables whole, and prepends a structured contextual header (document, section path, scope, version and entities) to each chunk before embedding, so that every relevant clause remains findable and attributable to the right entity during aggregation.

### 2.2.3 Corpus Partitioning and Domain Scoping

Semantic similarity is not always an adequate basis for organising a large collection. Maio and Rizzi [29] approach corpus organisation through concepts from analytical data management: their position paper adapts the Dimensional Fact Model, using dimensions such as time and organisational context to partition the corpus and route retrieval. This perspective suits enterprise collections, where a useful search boundary may be defined by business unit, reporting period or another operational attribute rather than by topic alone. Subedi et al. [37] study what happens as an index grows: passages that are close in meaning but belong to the wrong context crowd out the right ones, which they term *vector search dilution*, and restricting search to the relevant domain through metadata reduces the problem. Both studies imply that increasing corpus size does not automatically improve retrieval, and that search boundaries may need to reflect domain or organisational context. In LensRAG, the scope, version and entity fields of each chunk header serve this purpose: per-entity sub-queries are restricted to the relevant country, policy version or document type, which limits dilution as the corpus grows from the core manual to its extended versions.

### 2.2.4 Agentic RAG and Query Decomposition

A further line embeds autonomous agents in the pipeline. Agents decompose a query into sub-tasks, choose tools, retrieve iteratively and verify intermediate results [36]. Recent systematisation work formalises agentic RAG as a partially observable sequential decision process and argues that static generation metrics are insufficient to evaluate it [30]. Reinforcement learning has also been used to train models to interleave reasoning with search calls [18].

Decomposition is the agentic step most relevant to aggregation. HiRAG separates the work into decomposition, definition, retrieval, filtering and summarisation, combines document-level sparse retrieval with chunk-level dense retrieval, and uses a rethink step to trigger another retrieval attempt when the evidence is insufficient [44]. Ammann et al. [1] first separate a complex question into smaller information needs, re-

trieve passages for each sub-question, merge the candidates and apply cross-encoder reranking; this makes implicit evidence requirements visible and improves coverage, at the cost of more retrieval operations and a larger candidate set. Petcu et al. [33] treat decomposition as a balance between exploring for additional evidence and exploiting what has already been found, using feedback about document usefulness to spend more effort on unresolved sub-questions. SParC-RAG coordinates sequential refinement and parallel exploration of distinct sub-queries over a shared global context, aiming to keep the benefits of exploration without letting the evidence set grow without control [43]. These studies show the value of focused, decomposed retrieval, but each additional query, tool call or reasoning step increases latency and cost. LensRAG uses a restricted form of decomposition: one sub-query per relevant entity, taken from an entity list built at ingestion, which keeps the number of sub-queries bounded and makes coverage checkable.

### 2.2.5 Retrieval Quality, Coverage and Evidence Selection

Several studies argue that retrieving more is not the same as retrieving well. Zheng et al. [47] argue that standard scores such as recall and Mean Reciprocal Rank say little about what each retriever adds when several are combined. Their benchmark, MIGRASCOPE, uses mutual-information measures to estimate how much a retriever overlaps with, complements or adds to the others; in their experiments dissimilar retrievers add complementary evidence, whereas closely related ones largely repeat each other. Peng et al. [32] show that high recall does not necessarily produce accurate answers, introduce the Recall Conversion Rate, and use evidence chains in NeocorRAG to remove distractions and make connections between facts explicit. TRACE converts retrieved documents into a knowledge graph and builds reasoning chains step by step, with a triple ranker proposing candidates and an LLM selecting the next triple [8]. On HotpotQA, 2WikiMultiHopQA and MuSiQue it improves average exact-match accuracy by 14.03 percentage points over using all retrieved documents, with much shorter answer context, although it relies on title-centred Wikipedia graph extraction and does not directly evaluate chain quality.

For aggregation queries, coverage becomes the primary requirement. PluriHopRAG is designed for questions where missing a single relevant document makes the answer wrong and where many near-miss documents are present [38]. Instead of stopping at the top-$k$, it asks a sub-question of each document and uses a cross-encoder to drop irrelevant ones before the LLM reasons over the rest. LensRAG follows the same principle of retrieving until no new relevant evidence appears, combines keyword and dense retrieval as complementary signals in the sense of Zheng et al. [47], and applies an LLM filter that keeps only passages satisfying the query condition, which addresses the precision concerns raised by Peng et al. [32].

### 2.2.6 Structured and Graph-Based RAG

Another line makes relationships within the collection explicit. GraphRAG extracts entities and relationships, detects hierarchical communities, pre-computes community summaries and combines community-level partial answers into a global response with a map–reduce step, shifting the retrieval target from isolated passages to connected groups of concepts [7]. The trade-off is the processing needed to extract entities, build

the graph and generate summaries. LeanRAG points out that community summaries can end up isolated from one another even when their topics overlap, a problem its authors call *semantic islands*. It therefore adds explicit links between summary-level concepts and retrieves from the fine-grained entities upwards through these links [45]. HippoRAG uses a graph as a memory structure and applies personalised PageRank to find connected knowledge, which helps when a question requires following a chain of associations [12, 13]. LightRAG combines graph-based organisation with vector retrieval, supports both entity-level and theme-level searches, and supports incremental updates [11]. Knowledge Graph Prompting represents both document content and document structure, such as pages and tables, as graph nodes and uses LLM-guided traversal to select material for multi-document question answering, showing that useful context may lie in a passage's position and relationship to other document components [40]. Jia et al. [16] extend graph RAG to heterogeneous academic graphs of authors, papers and venues, with agents for query-aware retrieval, evidence expansion and graph-grounded verification.

Graph retrieval is not universally better, however. Han et al. [14] compare conventional RAG and GraphRAG on factual questions, multi-hop reasoning and query-focused summarisation, and find that passage-based RAG often fits narrowly defined factual lookups, while graph-oriented methods help when the task depends on broader connections; retrieval should therefore be selected according to the question. StructRAG follows this logic by routing each task to the most suitable structure type (for example a table, graph or catalogue), converting the documents into that structure at inference time and answering from it, with the largest gains when information is highly scattered [22].

Structured RAG (S-RAG) is the closest structured approach to aggregation: during indexing it turns the documents into database records, and at query time it answers questions by generating SQL over those records [19]. This gives exact answers when the schema covers the question, but requires the schema to be defined and populated in advance. Summary-based methods such as GraphRAG suit thematic questions but lose exact values during summarisation. LensRAG takes elements of both: like StructRAG and Han et al. [14], it routes by query type; like GraphRAG, it uses map–reduce for theme and sentiment queries; but unlike S-RAG, it builds no schema or database over the corpus.

### 2.2.7 Knowledge Refinement and Structured Reasoning

A relevant passage can still contain details that do not help answer the current question, so other work refines retrieved knowledge before generation. RankCoT combines ranking signals with reasoning-based summarisation, using preference training and self-reflection to produce concise, relevant evidence [41]. CoT-RAG combines knowledge-graph-guided reasoning, case-aware retrieval and structured pseudo-program prompts, using intermediate reasoning steps to organise the information need and retrieve relevant cases for each part of the task [21]. These approaches improve how evidence is used, but depend respectively on reliable training examples and on retrieved cases and graph relationships that adequately support the answer. LensRAG instead refines evidence into short query-time records of (entity, value, evidence span), which need no training and keep every value traceable to its source.

### 2.2.8 Evidence Verification and Conflict Handling

A dependable system must recognise weak or conflicting evidence rather than treating every retrieved passage as trustworthy. Self-RAG learns, through reflection tokens, to assess the relevance of retrieved passages and to judge whether a generated response is supported by them, making evidence assessment part of generation rather than an afterthought [4]. Corrective RAG (CRAG) adds a lightweight retrieval evaluator that grades the retrieved results and, when they are unreliable or insufficient, refines them or consults additional sources before generating an answer [42]. EG-RAG goes a step further and builds a graph over the evidence itself [15]: pairs of relevant sentences are labelled by a natural language inference model as agreeing, disagreeing or unrelated, and the resulting weighted links let the system tell corroboration apart from conflict. This is particularly pertinent to large corpora, where documents may report incompatible claims or reflect different versions of the same information. Policy corpora are a clear case: a value may differ between a global chapter, a country addendum and an older policy version. LensRAG's verification step addresses this by checking completeness against the entity list, flagging conflicting values from different sections or versions, and confirming that every statement in the answer is backed by an evidence span.

### 2.2.9 Corpus-Level Aggregation: Benchmarks and Methods

The work closest to this project explicitly targets aggregation over text. Luo et al. [26] introduce GlobalQA, a benchmark of counting, extremum, sorting and top-$k$ queries that require evidence from many documents. On this benchmark the best existing RAG baseline scores only 1.51 F1. Their own system, GlobalRAG, keeps documents intact as retrieval units, removes irrelevant documents with an LLM-based filter and leaves the final counting or ranking to symbolic tools; with Qwen2.5-14B it reaches 6.63 F1. The same group's MARAG-R1 moves beyond a single retriever and a fixed top-$k$ result set: an LLM agent coordinates semantic search, keyword search, document filtering and result aggregation, and supervised fine-tuning followed by reinforcement learning trains its decisions about which tools to invoke over multiple steps [27]. Zhu et al. [48] treat aggregation as a task in which the answer must be grounded in a fixed corpus and every qualifying entity must be found. Their benchmark, AGGBench, is drawn from finance reports and research papers and checked by human annotators, and their DFA baseline splits the work into three stages (resolving what the query refers to, filtering candidates, then aggregating), which finds more of the required evidence than RAG and agentic baselines. CorpusQA pushes the corpus size to 10 million tokens; its answers are produced by code rather than annotators, by turning documents into tables and running SQL over them [25]. It also finds that long-context models lose accuracy quickly as the corpus grows. Together these works establish that aggregation queries are largely unsolved, and that completeness of evidence and exact computation are the central difficulties. LensRAG shares GlobalRAG's filter-then-compute design and DFA's emphasis on completeness; unlike MARAG-R1, it requires no model training, and it adds context-enriched indexing and explicit verification.

### 2.2.10 LLM-Powered Analytics over Unstructured Documents

The database community approaches the same problem as query processing over documents. Early work such as EVAPORATE extracts structured views from semi-structured documents with LLMs [3]. LOTUS defines *semantic operators*: relational operations such as filter, join, rank and aggregate whose conditions are written in natural language and evaluated by an LLM, so that they can be chained like database queries [31]. Its authors note that RAG and Text-to-SQL suit only analyses whose evidence fits in one model call. DocETL optimises LLM document-processing pipelines through agent-driven rewrites [35], and Palimpzest provides cost-aware declarative optimisation of such pipelines [23]. Aryn combines layout-aware parsing with Luna, a planner that translates natural-language questions into inspectable query plans, and reports better accuracy than RAG on analytic queries over accident reports [2]. UDA-Bench provides a large benchmark for this setting, with over 100,000 documents and 240 queries spanning selection, filtering, aggregation and joins [6]. This literature shows that LLM extraction followed by exact computation outperforms free-text reasoning for analytic queries. LensRAG adopts that principle but keeps it lightweight: values are extracted only for the passages a query needs, held as temporary records and computed in code, rather than materialised as tables over the whole corpus.

### 2.2.11 Long-Context Models as an Alternative

Million-token context windows raise the question of whether retrieval is needed at all for small corpora. Evidence suggests that long context does not remove the problem: models under-use information in the middle of long inputs [24], and on corpus-level analysis tasks accuracy falls substantially as context length grows from 128K to 1M tokens [25]. StructRAG likewise outperforms a long-context baseline on knowledge-intensive tasks [22]. Long-context prompting therefore remains an essential *baseline*, particularly when a corpus fits in the window, as the core BragsLabs policy manual does, but not a general solution.

## 2.3 Comparative Discussion

Taken together, the studies suggest that corpus-level RAG is not a single retrieval technique but a set of interdependent design choices. A first group changes how information is organised before a question is asked: RAPTOR, Ψ-RAG and HiRAG give access to information at different levels of abstraction, LeanRAG links related summaries, and OLAP-inspired partitioning and domain scoping define organisational boundaries for retrieval. A second group makes relationships explicit: GraphRAG, HippoRAG, LightRAG, Knowledge Graph Prompting and heterogeneous graph RAG connect entities, passages or document components, although the comparison by Han et al. [14] cautions against treating graph retrieval as a universal replacement for passage search. A third group decides how retrieval should proceed: decomposition, exploration–exploitation, SParC-RAG and multi-tool agents such as MARAG-R1 aim to broaden coverage where it is needed. A fourth group assesses the evidence passed to the generator: reranking and information-theoretic evaluation judge retrieved candidates, RankCoT and NeocorRAG refine how information is represented,

and Self-RAG, CRAG and EG-RAG detect unreliable or conflicting evidence. Finally, aggregation-focused work (GlobalRAG, DFA, CorpusQA and database-style systems) shows that exact answers require exhaustive evidence and computation outside the LLM.

These choices involve trade-offs. Hierarchies provide compact navigation but may obscure links between branches. Graphs make relationships explicit but require construction and maintenance. Adaptive agents can broaden or refine a search but consume more time and computation. Evidence filtering reduces noise, yet overly aggressive filtering risks discarding a passage that is necessary in combination with others. A practical architecture therefore needs to select and coordinate these capabilities according to the question, the corpus and the required level of assurance, which is the approach taken by LensRAG's query router.

## 2.4 Gap Analysis

Table 2.1 summarises how representative prior work relates to the requirements of this project. Five gaps stand out.

1. **Domain.** Existing aggregation benchmarks are built from encyclopaedic, academic, financial or incident-report corpora. Policy documents have been studied for local question answering [28], but to our knowledge no work evaluates aggregation queries over enterprise policy, where answers depend on jurisdiction, versioning, exceptions and cross-references between chapters.
2. **Exact aggregation without SQL.** Methods that compute exact answers usually build tables or a schema over the corpus in advance [19, 25, 31], whereas schema-free methods compute less reliably. Few works obtain exact counts and rankings from a plain text and vector index, with structure created only at query time.
3. **Context and scope at indexing.** Context-enriched embeddings [10] and metadata-based partitioning [29, 37] have been studied separately, mainly for local retrieval. They have not been combined as a means of making every relevant clause findable, correctly scoped and attributable for aggregation queries.
4. **Evidence completeness and conflicts across versions.** Passage relevance does not establish that every entity in an aggregation has been covered. Conflict-aware methods such as EG-RAG [15] detect contradictions between statements, but few systems verify completeness or distinguish a true conflict from a legitimate policy update across versions.
5. **Consistent evaluation.** Prior studies use different tasks and measures. Few report aggregation accuracy, retrieval recall, faithfulness, cost and latency together, or test both local lookup and corpus-level aggregation on the same corpus.

## 2.5 Positioning of This Work

This project extends corpus-level aggregation [26, 48] to a new domain, enterprise HR policy, through the LensRAG method described in Section 1.1.1. Each of its components builds on established work: structure-aware, context-enriched chunk embeddings [10, 28], metadata scoping of retrieval [29, 37], query routing by

**Table 2.1.** Positioning of representative prior work against the requirements of this project. Entries summarise each work's primary focus as described by its authors.

| **Work** | **Target queries** | **How answers are computed** | **Corpus / domain** |
|---|---|---|---|
| GraphRAG [7] | Global, thematic sense-making | Map–reduce over community summaries | Podcasts, news |
| StructRAG [22] | Knowledge-intensive, scattered | Task-specific structures built at inference | General QA benchmarks |
| S-RAG [19] | Aggregative questions | Schema built at ingestion; SQL queries | Aggregative QA datasets |
| GlobalRAG [26] | Count, extremum, sort, top-$k$ | LLM filtering + symbolic aggregation | Encyclopaedic corpus |
| MARAG-R1 [27] | Corpus-level QA | RL-trained multi-tool retrieval agent | GlobalQA |
| DFA / AGGBench [48] | Entity-level aggregation, completeness | Disambiguate–filter–aggregate agent | Finance reports, papers |
| PluriHopRAG [38] | Exhaustive, recall-sensitive QA | Per-document sub-questions + cross-encoder filter | Distractor-rich corpora |
| EG-RAG [15] | Multi-document reasoning | NLI-based evidence graph (support/contradict) | Multi-document QA |
| LOTUS [31] | Bulk semantic analytics | Declarative semantic operators | Various |
| **LensRAG (this project)** | Local and aggregation queries, incl. themes, sentiment and conflicts | Context-enriched, scoped embeddings; per-entity coverage retrieval; query-time extraction; computation in code or map–reduce; verification; no SQL | Enterprise HR policy (multi-country, multi-version) |

type [14, 22], entity-level decomposition and coverage-oriented retrieval [1, 38, 43], LLM filtering followed by symbolic aggregation [26], map–reduce summarisation for thematic queries [7], and evidence verification with conflict detection [15, 42]. To our knowledge, the combination has not been evaluated on aggregation queries, and it differs from S-RAG [19] and database-style systems [31] in that no schema or SQL database is built over the corpus, and from MARAG-R1 [27] in that it requires no model training. Following Lu et al. [25], ground truth for aggregation queries is derived independently of the systems under test so that results are verifiable, and a long-context baseline is included because the core policy manual fits within current context windows. The evaluation reports aggregation accuracy, retrieval recall, faithfulness, cost and latency together, and adds completeness and conflict-detection measures that prior benchmarks do not report.

# 3

# Methodology

## 3.1 Research Design

Experimental, design-science, survey or mixed; justify the choice.

## 3.2 System Architecture

Provide an architecture diagram (Figure 3.1) and describe each component, the technology stack (frameworks, models, vector stores, protocols) and the reasons for each choice.

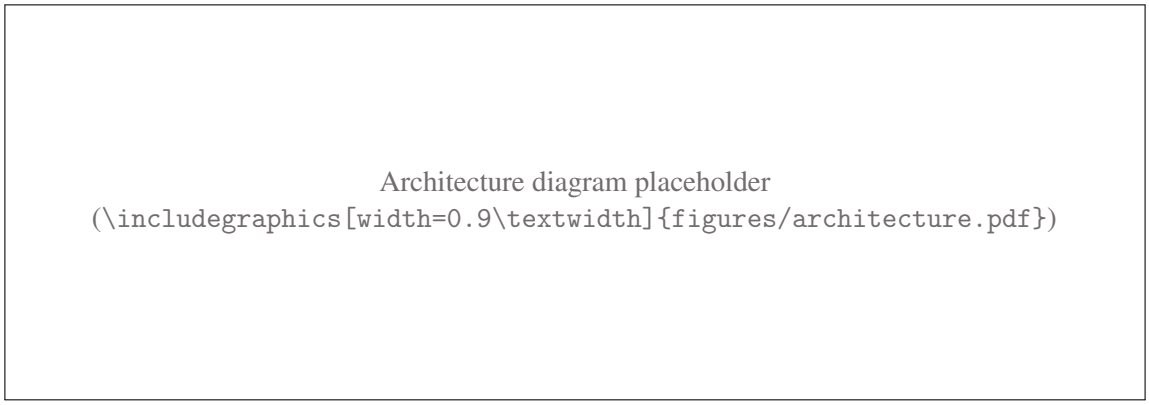



**Figure 3.1.** System architecture.

Table 3.1 lists candidate models for each stage of the LensRAG pipeline (Figure 1.1). High-volume, per-passage steps (classification, sub-query generation, filtering and extraction) use small, inexpensive models, while a stronger model is reserved for map–reduce summarisation and final answer generation. The models actually used, with exact version identifiers and access dates, are recorded in Chapter 4.

## 3.3 Data and Benchmarks

Datasets, benchmark suites, task construction, annotation protocol and inter-annotator agreement; licences; contamination controls.

**Table 3.1.** Candidate models for each stage of the LensRAG query pipeline. Final choices and exact versions are reported in Chapter 4.

| **Stage (model role)** | **Open-source / self-hosted** | **API / hosted** |
|---|---|---|
| Classify query type *(small LLM classifier)* | Qwen2.5-3B-Instruct or Phi-4-mini (few-shot); or a fine-tuned DeBERTa-v3 / SetFit classifier | GPT-4o-mini, Claude Haiku, Gemini Flash |
| Search + rerank *(hybrid + reranker)* | BM25 (OpenSearch / Elasticsearch) + BGE-M3 embeddings; bge-reranker-v2-m3 | OpenAI text-embedding-3-large or Voyage embeddings; Cohere Rerank |
| Select relevant entities *(rules + LLM)* | Lookup over the ingestion-time entity list + spaCy NER; small LLM fallback | Same rules; GPT-4o-mini or Claude Haiku fallback |
| Per-entity adaptive retrieval *(embedder + BM25 + LLM)* | Same hybrid index; Qwen2.5-7B-Instruct for sub-queries and stopping | Same hybrid index; GPT-4o-mini or Claude Haiku |
| LLM filter + extract *(fast LLM, JSON output)* | Qwen2.5-7B/14B-Instruct or Llama-3.1-8B-Instruct with JSON-constrained decoding (vLLM / Ollama) | GPT-4o-mini (structured outputs); Claude Haiku (tool-use JSON) |
| Compute in code *(no model)* | Python / pandas (count, max, sort, top-$k$) | – |
| Map–reduce summarise *(strong LLM)* | Qwen2.5-72B-Instruct or Llama-3.3-70B-Instruct; optional RoBERTa sentiment classifier for per-record labels | GPT-4o, Claude Sonnet, Gemini Pro |
| Verify evidence *(LLM judge + rules)* | Rule checks (entity coverage, cross-version values); DeBERTa-v3-large NLI; Qwen2.5-14B judge | Rule checks; GPT-4o or Claude Sonnet judge; RAGAS faithfulness |
| Generate answer *(strong LLM + citations)* | Qwen2.5-72B-Instruct or Llama-3.3-70B-Instruct | GPT-4o, Claude Sonnet, Gemini Pro |

## 3.4 Evaluation Protocol

Metrics (accuracy, pass^k, latency p50/p95, tokens and USD per successful task, attack-success rate, etc.); baselines; number of seeds; statistical tests; judge-model choice and its validation against humans.

## 3.5 Cost, Governance and Observability Instrumentation

Describe how cost was tracked (token accounting, GPU hours), how traces were captured (OpenTelemetry, LangSmith, Langfuse), and which governance or safety controls were evaluated. *This section is mandatory for all projects.*

## 3.6 Threats to Validity

Construct, internal, external and conclusion validity.

# 4 Implementation

## 4.1 Development Environment and Reproducibility

Repository layout, environment specification, seeds, model versions and API snapshots (record exact model identifiers and dates).

## 4.2 Key Components

Describe non-obvious engineering decisions. Use listings sparingly.

**Listing 4.1.** Illustrative LangGraph node definition.

```
def retrieval_evaluator(state: AgentState) -> AgentState:
    """Grade retrieved passages; route to refine / web-search / generate."
       ""
    scores = grader.batch(state["documents"], state["question"])
    state["route"] = "generate" if max(scores) > TAU else "web_search"
    return state
```

## 4.3 Deployment

Serving stack, autoscaling, monitoring, rollback; edge or on-premise constraints where relevant.

# 5 Results

## 5.1 Main Findings

Answer each RQ directly with tables and figures. Report confidence intervals or variance across seeds.

**Table 5.1.** Main results (illustrative). Best value per column in bold.

| Configuration | Accuracy (%) | p95 latency (s) | USD / task | Attack success (%) |
|---|---|---|---|---|
| Baseline | 71.2 | 3.4 | 0.041 | 61.0 |
| Variant A | 74.8 | 4.1 | 0.052 | 22.5 |
| Variant B (ours) | **75.1** | 4.4 | 0.055 | **4.1** |

## 5.2 Ablations and Sensitivity

## 5.3 Cost–Performance Trade-offs

Present the Pareto frontier (accuracy vs. cost) — a required figure.

## 5.4 Error Analysis

Qualitative taxonomy of failures with counts.

# 6 Discussion

## 6.1 Interpretation

## 6.2 Implications for Practitioners

Concrete recommendations; a decision framework if applicable.

## 6.3 Implications for Research

## 6.4 Limitations

## 6.5 Ethical, Legal and Societal Considerations

Privacy, safety, regulatory mapping (EU AI Act, DPDP, sector rules) where relevant.

# 7

# Conclusion and Future Work

## 7.1 Summary of Contributions

## 7.2 Future Work

## 7.3 Publication Plan

Target venue(s), arXiv category (e.g., cs.AI, cs.CL, cs.SE), and the condensed paper's structure.

# Bibliography


[1] Paul J. L. Ammann, Jonas Golde, and Alan Akbik. Question decomposition for retrieval-augmented generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 4: Student Research Workshop)*, pages 497–507, Vienna, Austria, 2025. Association for Computational Linguistics. URL `https://aclanthology.org/2025.acl-srw.32`.

[2] Eric Anderson, Jonathan Fritz, Austin Lee, Bohou Li, Mark Lindblad, Henry Lindeman, Alex Meyer, Parthkumar Parmar, Tanvi Ranade, Mehul A. Shah, Benjamin Sowell, Dan Tecuci, Vinayak Thapliyal, and Matt Welsh. The design of an LLM-powered unstructured analytics system. In *Conference on Innovative Data Systems Research (CIDR)*, 2025. arXiv:2409.00847.

[3] Simran Arora, Brandon Yang, Sabri Eyuboglu, Avanika Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. Language models enable simple systems for generating structured views of heterogeneous data lakes. *Proceedings of the VLDB Endowment*, 17(2), 2023. arXiv:2304.09433.

[4] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.11511.

[5] Scott Barnett, Stefanus Kurniawan, Srikanth Thudumu, Zach Brannelly, and Mohamed Abdelrazek. Seven failure points when engineering a retrieval augmented generation system. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering – Software Engineering for AI (CAIN)*, 2024. arXiv:2401.05856.

[6] Qiyan Deng, Jianhui Li, Chengliang Chai, Ye Yuan, et al. Unstructured data analysis using LLMs: A comprehensive benchmark. *Proceedings of the VLDB Endowment*, 19(9):2398–2410, 2026. arXiv:2510.27119.

[7] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[8] Jinyuan Fang, Zaiqiao Meng, and Craig MacDonald. TRACE the evidence: Constructing knowledge-grounded reasoning chains for retrieval-augmented generation. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 8472–8494, Miami, Florida, USA, 2024. Association for Computational Linguistics.

[9] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.

[10] Michael Günther, Isabelle Mohr, Daniel James Williams, Bo Wang, and Han Xiao. Late chunking: Contextual chunk embeddings using long-context embedding models. *arXiv preprint arXiv:2409.04701*, 2024.

[11] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. LightRAG: Simple and fast retrieval-augmented generation. *arXiv preprint arXiv:2410.05779*, 2024.

[12] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.14831.

[13] Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. From RAG to memory: Non-parametric continual learning for large language models. *arXiv preprint arXiv:2502.14802*, 2025. HippoRAG 2.

[14] Haoyu Han, Li Ma, Yu Wang, Harry Shomer, Yongjia Lei, Zhisheng Qi, Kai Guo, Zhigang Hua, Bo Long, Hui Liu, Charu C. Aggarwal, and Jiliang Tang. RAG vs. GraphRAG: A systematic evaluation and key insights. *arXiv preprint arXiv:2502.11371*, 2025.

[15] Seunggwan Hong, Junhyung Moon, Eunkyeong Lee, Jaehyoung Park, and Hyunseung Choo. EG-RAG: Retrieval-augmented generation with evidence graph for reliable multi-document reasoning. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 1838–1846, Paphos, Cyprus, 2026. IFAAMAS. doi: 10.65109/NJIG6104.

[16] Runsong Jia, Mengjia Wu, Ying Ding, Jie Lu, and Yi Zhang. Agent-enhanced heterogeneous graph RAG for academic question answering. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*. ACM, 2026. doi: 10.1145/3774904.3792960. arXiv:2609.00761.

[17] Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of EMNLP*, 2023. arXiv:2305.06983.

[18] Bowen Jin et al. Search-R1: Training LLMs to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.

[19] Omri Koshorek, Niv Granot, Aviv Alloni, Shahar Admati, Roee Hendel, Ido Weiss, Alan Arazi, Shay-Nitzan Cohen, and Yonatan Belinkov. Structured RAG for answering aggregative questions. *arXiv preprint arXiv:2511.08505*, 2025.

[20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2005.11401.

[21] Feiyang Li, Peng Fang, Zhan Shi, Arijit Khan, Fang Wang, Weihao Wang, Zhangxin-hw, and Yongjian Cui. CoT-RAG: Integrating chain of thought and retrieval-augmented generation to enhance reasoning in large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 3119–3171, Suzhou, China, 2025. Association for Computational Linguistics.

[22] Zhuoqun Li, Xuanang Chen, Haiyang Yu, Hongyu Lin, Yaojie Lu, Qiaoyu Tang, Fei Huang, Xianpei Han, Le Sun, and Yongbin Li. StructRAG: Boosting knowledge intensive reasoning of LLMs via inference-time hybrid information structurization. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.08815.

[23] Chunwei Liu et al. Palimpzest: Optimizing AI-powered analytics with declarative query processing. In *Conference on Innovative Data Systems Research (CIDR)*, 2025.

[24] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 2024. arXiv:2307.03172.

[25] Zhiyuan Lu, Chenliang Li, Yingcheng Shi, Weizhou Shen, Ming Yan, and Fei Huang. CorpusQA: A 10 million token benchmark for corpus-level analysis and reasoning. *arXiv preprint arXiv:2601.14952*, 2026.

[26] Qi Luo, Xiaonan Li, Tingshuo Fan, Xinchi Chen, and Xipeng Qiu. Towards global retrieval augmented generation: A benchmark for corpus-level reasoning. *arXiv preprint arXiv:2510.26205*, 2025.

[27] Qi Luo, Xiaonan Li, Yuxin Wang, Tingshuo Fan, Yuan Li, Xinchi Chen, and Xipeng Qiu. MARAG-R1: Beyond single retriever via reinforcement-learned multi-tool agentic retrieval. *arXiv preprint arXiv:2510.27569*, 2025.

[28] Anuj Maharjan and Umesh Yadav. Chunking, retrieval, and re-ranking: An empirical evaluation of RAG architectures for policy document question answering. *arXiv preprint arXiv:2601.15457*, 2026.

[29] Dario Maio and Stefano Rizzi. Bridging OLAP and RAG: A multidimensional approach to the design of corpus partitioning. *arXiv preprint arXiv:2601.03748*, 2026. Position paper.

[30] Saroj Mishra, Suman Niroula, Umesh Yadav, Dilip Thakur, Srijan Gyawali, and Shiva Gaire. SoK: Agentic retrieval-augmented generation (RAG): Taxonomy, architectures, evaluation, and research directions. *arXiv preprint arXiv:2603.07379*, 2026.

[31] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. Semantic operators and their optimization: Enabling LLM-based data processing with accuracy guarantees in LOTUS. *Proceedings of the VLDB Endowment*, 18(11):4171–4184, 2025. arXiv:2407.11418.

[32] Shiyao Peng, Qianhe Zheng, Zhuodi Hao, Zichen Tang, Rongjin Li, Qing Huang, Jiayu Huang, Jiacheng Liu, Yifan Zhu, and Haihong E. NeocorRAG: Less irrelevant information, more explicit

evidence, and more effective recall via evidence chains. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, Dubai, United Arab Emirates, 2026. ACM. doi: 10.1145/3774904.3792093. arXiv:2604.27852.

[33] Roxana Petcu, Kenton Murray, Daniel Khashabi, Evangelos Kanoulas, Maarten de Rijke, Dawn Lawrie, and Kevin Duh. Query decomposition for RAG: Balancing exploration-exploitation. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6857–6871, Rabat, Morocco, 2026. Association for Computational Linguistics. URL `https://aclanthology.org/2026.eacl-long.322/`.

[34] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2401.18059.

[35] Shreya Shankar et al. DocETL: Agentic query rewriting and evaluation for complex document processing. *Proceedings of the VLDB Endowment*, 2025. arXiv:2410.12189.

[36] Aditi Singh, Abul Ehtesham, Saket Kumar, Tala Talaei Khoei, and Athanasios V. Vasilakos. Agentic retrieval-augmented generation: A survey on agentic RAG. *arXiv preprint arXiv:2501.09136*, 2025. v4, April 2026.

[37] Nabaraj Subedi, Ahmed Abdelaty, and Shivanand Venkanna Sheshappanavar. When more documents hurt RAG: Measuring and reducing vector search dilution with domain-scoped retrieval. *arXiv preprint arXiv:2606.11350*, 2026.

[38] Mykolas Sveistrys and Richard Kunert. PluriHop: Exhaustive, recall-sensitive QA over distractor-rich corpora. *arXiv preprint arXiv:2510.14377*, 2025.

[39] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of ACL*, 2023. arXiv:2212.10509.

[40] Yu Wang, Nedim Lipka, Ryan A. Rossi, Alexa Siu, Ruiyi Zhang, and Tyler Derr. Knowledge graph prompting for multi-document question answering. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19206–19214, 2024. doi: 10.1609/aaai.v38i17.29889.

[41] Mingyan Wu, Zhenghao Liu, Yukun Yan, Xinze Li, Shi Yu, Zheni Zeng, Yu Gu, and Ge Yu. RankCoT: Refining knowledge for retrieval-augmented generation through ranking chain-of-thoughts. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12857–12874, Vienna, Austria, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.acl-long.629.

[42] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation. *arXiv preprint arXiv:2401.15884*, 2024.

[43] Yuxin Yang, Gangda Deng, Ömer Faruk Akgül, Nima Chitsazan, Yash Govilkar, Akasha Tigalappanavara, Shi-Xiong Zhang, Sambit Sahu, and Viktor Prasanna. SParC-RAG: Adaptive sequential–parallel scaling with context management for retrieval-augmented generation. *arXiv preprint arXiv:2602.00083*, 2026. Check whether the EMNLP 2026 version (aclanthology.org/2026.emnlp-main.1865) should be cited instead.

[44] Xiaoming Zhang, Ming Wang, Xiaocui Yang, Daling Wang, Shi Feng, and Yifei Zhang. Hierarchical retrieval-augmented generation model with rethink for multi-hop question answering. *arXiv preprint arXiv:2408.11875*, 2024.

[45] Yaoze Zhang, Rong Wu, Pinlong Cai, Xiaoman Wang, Guohang Yan, Song Mao, Ding Wang, and Botian Shi. LeanRAG: Knowledge-graph-based generation with semantic aggregation and hierarchical retrieval. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026. URL `https://ojs.aaai.org/index.php/AAAI/article/view/40789`. arXiv:2508.10391.

[46] Ziwen Zhao and Menglin Yang. Hierarchical abstract tree for cross-document retrieval augmented generation. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 162579–162609. PMLR, 2026. URL `https://proceedings.mlr.press/v306/zhao26bk.html`. Introduces Ψ-RAG.

[47] Wenqing Zheng, Dmitri Kalaev, Noah Fatsi, Daniel Barcklow, Owen Reinert, Igor Melnyk, Senthil Kumar, and C. Bayan Bruss. Revisiting RAG retrievers: An information theoretic benchmark. In *Proceedings of the 11th Workshop on Automated Knowledge Base Construction (AKBC)*, pages 393–408, Budapest, Hungary, 2026. Association for Computational Linguistics. URL `https://aclanthology.org/2026.akbc-1.27/`. Introduces MIGRASCOPE; preprint arXiv:2602.21553.

[48] Haojia Zhu et al. Benchmarking aggregation queries over unstructured text. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2026. URL `https://aclanthology.org/2026.emnlp-main.1026/`. Preprint: arXiv:2602.01355.

# A

# Individual Contribution Statement

| Member | Contribution (CRediT taxonomy) |
|---|---|
| Member One | Conceptualisation, methodology, writing — original draft |
| Member Two | Software, architecture, deployment |
| Member Three | Data curation, evaluation, visualisation |
| Member Four | Security testing, governance mapping, validation |
| Member Five | LLMOps, cost analysis, writing — review & editing |

# B

# Weekly Progress Log

| Week | Planned | Achieved | Advisor notes |
|---|---|---|---|
| 1 | Kick-off; Closing Project Title and Project Scope, Start of literature scan | | |
| 2 | 25 research articles research complete | | |
| 3 | 50 research articles research complete | | |
| 4 | Introduction and Literature Survey Complete | | |
| 5-6 | Getting Foundations for the Project Correct | | |
| 7 | Foundations Complete | | |
| 8–10 | Experiments | | |
| 11–12 | Analysis | | |
| 13–14 | Writing; arXiv submission | | |

# C

# Reproducibility Checklist

- ☐ Code and configuration released under an open licence at `https://github.com/brags-labs/corpusrag`.
- ☐ Exact model identifiers, versions and access dates recorded.
- ☐ Random seeds and number of runs reported for every result.
- ☐ Datasets released or access instructions provided; licences stated.
- ☐ Cost (tokens, USD, GPU-hours) reported per experiment.
- ☐ Prompts and judge instructions included verbatim in the appendix.
- ☐ Human-annotation protocol and agreement statistics reported.

# D

# Prompts, Judge Instructions and Additional Results

Include all prompts verbatim, extended tables and figures.